

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CODING GOLF COMPETITION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 1 – Coding Golf Competition
Weightage:	35% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	DHIVYADHARSAN M.S	Reg. No.:	192511465
Section / Batch:	2025	Date:	17/08/2024

Code of Conduct

I __DHIVYADHARSAN M.S_____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: DHIVYADHARSAN M.S_

Instructions

- Log in to the assigned coding platform - Code chef (<https://www.codechef.com/skill-test/c-language-dsa>). Total: 100 Marks.
- You can only attempt this assessment once. Also, you will not be able to pause the assessment after starting.
- You are not allowed to switch tabs during the assessment, otherwise you will be blocked from the assessment.
- You will get a detailed report on your performance at the end of the assessment.

Section / Topic	Focus	Q Nos	Marks
Data structures	Code Chef	28	100
GRAND TOTAL:		100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Accurately interprets all problem requirements and constraints before coding.	Understands most requirements with minor omissions.	Partially understands the problem; misses some constraints.	Misinterprets the problem or ignores key requirements.
Correctness of Solution	30	Program compiles successfully and passes all hidden and visible test cases.	Program passes most test cases with minor errors.	Program passes some test cases but has logical issues.	Program fails to compile or produces incorrect results.
Data Structure & Algorithm Selection	20	Selects the most appropriate data structure and algorithm with an optimized solution.	Chooses an appropriate solution with minor inefficiencies.	Uses a workable but less efficient approach.	Uses an inappropriate data structure or algorithm.
Code Quality & Documentation	20	Well-structured, modular, readable, and properly indented code with meaningful identifiers.	Readable code with minor formatting issues.	Basic coding style with inconsistent formatting.	Poorly organized code that is difficult to understand.
Time & Space Optimization	10	Demonstrates excellent optimization and correctly analyses time and space complexity.	Good optimization with minor inefficiencies.	Limited optimization and incomplete complexity analysis.	No optimization and incorrect or missing complexity analysis.

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – CODING GOLF COMPETITION

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 1 – Coding Golf Competition
Weightage:	35% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	17/08/2024

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Log in to the assigned coding platform - Code chef (<https://www.codechef.com/skill-test/c-language-dsa>). Total: 100 Marks.
- You can only attempt this assessment once. Also, you will not be able to pause the assessment after starting.
- You are not allowed to switch tabs during the assessment, otherwise you will be blocked from the assessment.
- You will get a detailed report on your performance at the end of the assessment.

Section / Topic	Focus	Q Nos	Marks
Data structures	Code Chef	28	100
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Accurately interprets all problem requirements and constraints before coding.	Understands most requirements with minor omissions.	Partially understands the problem; misses some constraints.	Misinterprets the problem or ignores key requirements.
Correctness of Solution	30	Program compiles successfully and passes all hidden and visible test cases.	Program passes most test cases with minor errors.	Program passes some test cases but has logical issues.	Program fails to compile or produces incorrect results.
Data Structure & Algorithm Selection	20	Selects the most appropriate data structure and algorithm with an optimized solution.	Chooses an appropriate solution with minor inefficiencies.	Uses a workable but less efficient approach.	Uses an inappropriate data structure or algorithm.
Code Quality & Documentation	20	Well-structured, modular, readable, and properly indented code with meaningful identifiers.	Readable code with minor formatting issues.	Basic coding style with inconsistent formatting.	Poorly organized code that is difficult to understand.
Time & Space Optimization	10	Demonstrates excellent optimization and correctly analyses time and space complexity.	Good optimization with minor inefficiencies.	Limited optimization and incomplete complexity analysis.	No optimization and incorrect or missing complexity analysis.

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Statement

Consider the following C code snippet:

```
void harmonicSeries(int n) {
    double sum = 0.0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            sum += 1.0 / j;
        }
    }
    printf("Sum: %f\n", sum);
}
```

What is the time complexity of the harmonicSeries function in terms of the input size n ? (Consider all the imports are included)

Select the correct answer

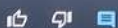
☐ A) $O(n)$

☒ B) $O(n^2)$

☐ C) $O(\log n)$

☐ D) $O(n \log n)$

Did you like the problem?
 14 users found this helpful



Submit

Next

FULL HD 1080

acer

Statement

You are given a string (S). Your task is to determine if the string can be a palindrome after deleting at most one character from it.

Input Format

- The first line contains one integer n , the size of the string - Next line contains string S .

Output Format

- Print whether S can be made palindrome after deleting at most one character.

Constraints

- $1 \leq |S| \leq 10^5$

Sample 1:

Input	Output
4 abca	true

Explanation:

We can delete b or c to make it palindrome. after deleting b the S will be aca which is palindrome.

Sample 2:

Input	Output
4 batr	false

C

```
1 #include <stdio.h>
2 #include <stdbool.h>
3 #include <string.h>
4
5 // Function to check if the string can be a palindrome after deleting at most one character
6 bool validPalindrome(const char* s) {
7     // Write your code here
8     int l=0, r=strlen(s)-1;
9
10    while(l<r){
11        if(s[l]!=s[r]){
12            int a=l+1, b=r;
13            while(a<b&&s[a]==s[b])a++,b--;
14            if(a==b)return true;
15            a=l, b=r-1;
16            while(a<b&&s[a]==s[b])a++,b--;
17            return a==b;
18        }
19        l++;
20        r--;
21    }
22    return true;
23 }
24
25 int main() {
26     int n;
27     scanf("%d", &n);
28     char s[100000];
29     scanf("%s", s); // Read the input string
30     printf("%s\n", validPalindrome(s) ? "true" : "false"); // Output the result
31     return 0;
32 }
```

Test against Custom Input

4
abca

Run

Submit

Next

FULL HD 1080p

acer

Statement

Consider the following C program:

```
struct MinQueue {
    int queue[100], minQueue[100], front, rear, minfront, minrear;
};

void enqueue(struct MinQueue* mq, int number) {
    mq->queue[mq->rear++] = number;
    while (mq->minrear > mq->minfront && mq->minQueue[mq->minrear - 1] > number) {
        mq->minrear--;
    }
    mq->minQueue[mq->minrear++] = number;
}

int dequeue(struct MinQueue* mq) {
    int number = mq->queue[mq->front++];
    if (number == mq->minQueue[mq->minfront]) {
        mq->minfront++;
    }
    return number;
}

int main() {
    struct MinQueue mq = { .front = 0, .rear = 0, .minfront = 0, .minrear = 0 };
    enqueue(&mq, 4);
    enqueue(&mq, 10);
    enqueue(&mq, 20);
    dequeue(&mq);
    enqueue(&mq, 5);
}
```

After executing the above operations, what is the current minimum value in the queue?

Did you like the problem?

5 users found this helpful



Select the correct answer

☐ A) 4

☒ B) 5

☐ C) 10

☐ D) 20

Submit

Clear

FULL HD 1080

acer

Statement

Given a string `num` representing a non-negative integer and an integer `k`, remove `k` digits from `num` to form the smallest possible integer. Return this smallest possible integer as a string.

Input Format

- First line contains two integers `n` and `k`, `n` is the size of `num` and `k` is the number of digits that need to be removed.
- Second line contains a string `num`.

Output Format

- Print the smallest possible integer as a string, without leading zero except for the zero itself.

Constraints

- $1 \leq n, k \leq 100000$
- `num` will not contain any leading zero except for the zero itself.

Sample 1:

Input	Output
5 2 91934	134

Explanation:

Remove both the 9 and the number formed will be 134.

Sample 2:

C

```

1 #include <stdio.h>
2 #include <string.h>
3
4 // Function to remove k digits from the input number
5 char* remove_k_digits(char* num, int k) {
6     // Write your code here
7     int n = strlen(num), top = 0;
8
9     for (int i = 0; i < n; i++) {
10         while (top > 0 && num[top-1] > num[i]) {
11             num[top-1] = num[i];
12             top--; k--;
13         }
14         num[top++] = num[i];
15     }
16
17     while (k > 0) {top--; k--;}
18
19     num[top] = '\0';
20
21     int i = 0;
22     while (i < top-1 && num[i] == '0') i++;
23
24     if (i > 0) memmove(num, num+i, top-i+1);
25     char* res = malloc(strlen(num)+1);
26     strcpy(res, num);
27     return res;
28 }
29
30 int main() {
31     int n, k;
32     scanf("%d %d", &n, &k);
    
```

Test against Custom Input

5 2
 91934

Ctrl + Enter

Run

Submit

Next

FULL HD 1080

acer

Statement

Consider the following C program:

```
#include <stdio.h>

int main() {
    int value1 = 5, value2 = 10;
    int *p1 = &value1, *p2 = &value2;
    int **dp1 = &p1, **dp2 = &p2;

    printf("Before modification:\n");
    printf("value1: %d, value2: %d\n", value1, value2);

    *dp1 = *dp2;
    **dp2 = 20;

    printf("After modification:\n");
    printf("value1: %d, value2: %d\n", value1, value2);
    printf("p1 points to: %d\n", *p1);
    printf("p2 points to: %d\n", *p2);

    return 0;
}
```

What will be the output?

Did you like the problem?
2 users found this helpful



Select the correct answer

☐ A) Before modification:
value1: 5, value2: 10
After modification:
value1: 5, value2: 20
p1 points to: 20
p2 points to: 20

☐ B) Before modification:
value1: 5, value2: 10
After modification:
value1: 20, value2: 10
p1 points to: 20
p2 points to: 10

☒ C) Before modification:
value1: 5, value2: 10
After modification:
value1: 10, value2: 20
p1 points to: 10
p2 points to: 20

☐ D) Before modification:
value1: 5, value2: 10

Out of 100%

Submit

Next

Statement

Given two integer arrays $nums1$ and $nums2$, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and return the result in sorted order.

Intersection is defined as the common element in both arrays. For example -

$nums1 = [1, 2]$ and $nums2 = [2, 1, 3]$

Therefore, only 1 and 2 are common in both arrays. On sorting the resultant array would be $[1, 2]$.

Input Format

- The first line contains two integers n, m — the size of $nums1$ and $nums2$.
- The second line contains n integers $a_1, a_2, \dots, a_n$ — the number of $nums1$.
- The third line contains m integers $b_1, b_2, \dots, b_m$ — the number of $nums2$.

Output Format

Print all the elements that are appearing in both the arrays in sorted order.

Constraints

- $1 \leq n, m \leq 100$
- $0 \leq a_i \leq 100$
- $0 \leq b_i \leq 100$

Sample 1:

Input	Output
2 3 1 2 2 1 3	1 2

```
1 int* intersect(int* nums1, int nums1Size, int* nums2, int nums2Size, int* returnSize) {
2     // write your code here
3     int* intersect(int* a, int n, int* b, int m, int* r){
4         static int c[100];
5         int i, j, k = 0;
6         for(j = 0; j < n; j++)
7             for(i = 0; i < m; i++)
8                 if(a[j] == b[i]){
9                     c[k++] = a[j];
10                    b[i] = -1;
11                    break;
12                }
13         *r = k;
14         return c;
15     }
16 }
17
18
19
```

Test against Custom Input

```
2 3
1 2
2 1 3
```

Clear

Run

Submit

Next

FULL HD 1080

acer

Statement:

What will be the output of the given code snippet:

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int *ptr = arr;

    printf("%d", *(ptr++));
    printf("%d", *(ptr + 2));
    printf("%d", *(++ptr));
    printf("%d", *(ptr--));

    return 0;
}
```

Select the correct answer

A) 1
4
3
2

B) 1
5
4
3

C) 1
4
4
3

D) 1
4
3
3

Did you find the problem?



Submit

Close

Statement

Given an array `arr` of positive integers and an integer `k`, write a program that returns the maximum possible sum of `k` elements taken from either the start of the array, the end of the array, or a combination of both.

Input Format

- The first line contains two positive integers `n` and `k` representing the size of `arr` and number of elements whose sum is to be taken.
- Next line contains `n` elements of the array.

Output Format

Print the maximum sum that can be achieved by taking `k` elements.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq k \leq n$
- $1 \leq arr[i] \leq 10^6$

Sample 1:

Input	Output
5 3 4 2 1 5 1	10

Explanation:

Take one element from the start of the array and take 2 elements from the end of the array, the sum will be $4+1+5=10$.

C

```

1 #include <stdio.h>
2
3 long long maxSumElements(long long arr[], int n, int k) {
4     // Write your code heregfh
5 }
6
7 int main() {
8     int n, k;
9     scanf("%d %d", &n, &k);
10    long long arr[n];
11    for (int i = 0; i < n; ++i) {
12        scanf("%lld", &arr[i]);
13    }
14
15    printf("%lld\n", maxSumElements(arr, n, k));
16    return 0;
17 }
18
    
```

Test against Custom Input

```

5 3
4 2 1 5 1
    
```

[Clear](#)
[Reset](#)

Run

Submit

Next

Saved



Senthannish Selvan

58 Min : 28 Sec Prev 9/30 Next

Statement

Consider the following C function:

```
int* nextGreaterElements(const int* nums, int numsSize, int* returnSize) {
    int* result = (int*)malloc(numsSize * sizeof(int));
    *returnSize = numsSize;
    Stack* s = createStack(numsSize);
    for (int i = 0; i < numsSize; ++i) {
        while (!isEmpty(s) && nums[s->array[s->top]] < nums[i % numsSize]) {
            result[s->array[s->top]] = nums[i % numsSize];
            pop(s);
        }
        if (i < numsSize) push(s, i);
    }
    free(s->array);
    free(s);
    return result;
}
```

What is the time complexity of the given nextGreaterElements function, assuming stack default operations are already implemented? (Consider all the imports are included)

Select the correct answer

☐ A) $O(n^2)$

☒ B) $O(n)$

☐ C) $O(n \log n)$

☐ D) $O(n^2 \log n)$

Did you like the problem?



Out of 16

Submit

Flag

FULL HD 1080

acer

Statement

- You are given a binary array `arr` whose elements are only 0 and 1. Your task is to find the length of the longest subarray that contains only 1s after flipping exactly one contiguous subarray from 0 to 1.
- You must perform the flip operation at least once.

Input Format

- The first line contains one integer `n`, the size of the array. - Next line contains `n` integers `arr[0], arr[1]...arr[n]`, representing the elements of the array.

Output Format

- Find length of longest subarray with 1 after flipping one contiguous subarray.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq arr[i] \leq 1$

Sample 1:

Input	Output
3 0 1 0	2

Explanation:

Flip the first contiguous 0 to make it 1, the arr would look like 1,1,0. it contains 2 ones that are maximum from all possible cases.

Sample 2:

```
1 #include <stdio.h>
2
3 int main() {
4     // Write your code here
5     hflth
6     return 0;
7 }
8
```

Test against Custom Input

```
3
0 1 0
```

Run

Submit

Next

Statement

Consider the following C function:

```
#include <stdio.h>

void mystery(int x) {
    printf("%d ", x);
    if (x % 2 == 0)
        mystery(x - 1);
    else
        mystery(x - 2);
}

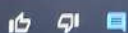
int main() {
    mystery(10);
    return 0;
}
```

What will be the output?

Select the correct answer

- ☐ A) It prints decreasing numbers starting from 10 and terminates normally.
- ☐ B) It prints decreasing even numbers starting from 10 and terminates normally.
- ☐ C) It prints decreasing even numbers starting from 10 and then causes a stack overflow.
- ☒ D) It results in an infinite recursion starting with the number 10 and eventually a stack overflow

Did you like the problem?
1 user found this helpful



Can't / No

Submit

Next

FULL HD 1080

acer

Statement

You are given an array `arr` of integers, which may contain duplicate elements. Your task is to write a recursive function to generate all unique permutations of this array. Each permutation should be unique, even if the array contains duplicate elements. The solution should not generate any duplicate permutations.

A permutation is a unique arrangement of elements where the order matters.

Input Format

- The first line contains one integer n , the size of the array. - Next line contains n integers `arr[0], arr[1]...arr[n]`, representing the elements of the array in the sorted order.

Output Format

- Generate the unique permutation in completing the `uniquePermutations` function.
- Printing the permutation will be done by the main function.

Constraints

- $1 \leq n \leq 10$.
- $1 \leq arr[i] \leq 10$.

Sample 1:

Input	Output
2	[1 2]
1 2	[2 1]

Explanation:

There are 2 permutations of [1 2] that are [1 2] and [2 1]

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 // Function to generate unique permutations
6 int **uniquePermutations(int *multiset, int multisetsize, int *returnSize, int
7     **returnColumnSizes) {
8     // Write your code here
9 }
10
11 // Function to print permutations
12 void printPermutations(int **permutations, int returnSize, int *returnColumnSizes) {
13     for (int i = 0; i < returnSize; ++i) {
14         printf("[");
15         for (int j = 0; j < returnColumnSizes[i]; ++j) {
16             printf("%d", permutations[i][j]);
17             if (j < returnColumnSizes[i] - 1) {
18                 printf(" ");
19             }
20             printf("]");
21         }
22         printf("\n");
23     }
24 }
25
26 int main() {
27     int n;
28     printf("Enter the number of elements: ");
29     scanf("%d", &n);
30
31     int *multiset = (int *)malloc(n * sizeof(int));
32     printf("Enter the elements: ");
33     for (int i = 0; i < n; ++i) {

```

Test against Custom Input

```

2
1 2

```

Ctrl + J

Run

Submit

Next

Statement

Given the following 2D matrix = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}}, what will be the value of result after calling fun(matrix, 3, 4)? (Consider all imports are included)

```
int* fun(int** arr, int row, int column) {
    int* result = (int*)malloc(row * column * sizeof(int));
    int k = 0, l = 0, index = 0;

    while (k < row && l < column) {
        for (int i = l; i < column; ++i) {
            result[index++] = arr[k][i];
        }
        k++;
        for (int i = k; i < row; ++i) {
            result[index++] = arr[i][column - 1];
        }
        column--;
        if (k < row) {
            for (int i = column - 1; i >= l; --i) result[index++] = arr[row - 1][i];
            row--;
        }
        if (l < column) {
            for (int i = row - 1; i >= k; --i) result[index++] = arr[i][l];
            l++;
        }
    }

    return result;
}
```

Select the correct answer

☐ A) 1 4 7 10 12 11 8 5 2 3 6 9

☐ B) 1 2 3 4 5 6 7 8 9 10 11 12

☒ C) 1 2 3 4 8 12 11 10 9 5 6 7

☐ D) 1 2 3 4 8 12 11 10 9 5 6 7 8

Did you like the problem?



Submit

Next

FULL HD 1080

acer

Statement

You are given an array `arr` consisting of only the integers 0, 1, and 2. Your task is to find the number of contiguous subarrays that contain at least one occurrence of each integer (0, 1, and 2).

Input Format

- The first line contains one integer n , the size of the array. - Next line contains n integers `arr[0], arr[1]...arr[n]`, representing the elements of the array.

Output Format

- Print the count of all subarrays that contains at least one occurrence of each integer (0, 1, 2).

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq arr[i] \leq 2$

Sample 1:

Input	Output
4 0 1 2 0	3

Explanation:

The subarrays are: [0, 1, 2], [0, 1, 2, 0], [1, 2, 0] Each subarray contains at least one 0, 1, 2.

Sample 2:

Input	Output
-------	--------

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 long long countSubarrays(long long* arr, long long n) {
5     // Write your code here
6     jtfjg
7 }
8
9 int main() {
10     long long n;
11     scanf("%lld", &n);
12
13     long long* arr = (long long*)malloc(n * sizeof(long long));
14     for (int i = 0; i < n; i++) {
15         scanf("%lld", &arr[i]);
16     }
17     printf("%lld\n", countSubarrays(arr, n));
18
19     free(arr);
20     return 0;
21 }
22 }
```

Test against Custom Input

```
4
0 1 2 0
```

Run

Submit

Next

FULL HD 1080

acer

Statement

Given the following array `int arr[] = {10, 80, 30, 90, 40, 50, 70};`

```
void helper(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int l1[n1], l2[n2];

    for (int i = 0; i < n1; i++) l1[i] = arr[l + i];
    for (int j = 0; j < n2; j++) l2[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l; // Initial indices

    while (i < n1 && j < n2) arr[k++] = (l1[i] <= l2[j]) ? l1[i++] : l2[j++];
    while (i < n1) arr[k++] = l1[i++];
    while (j < n2) arr[k++] = l2[j++];
}

void fun(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        fun(arr, l, m);
        fun(arr, m + 1, r);
        helper(arr, l, m, r);
    }
}
```

What will be the output after calling `fun(arr, 0, 6)?` (Consider all inputs are included)

Select the correct answer

☐ A) 10 80 30 90 40 50 70

☒ B) 10 30 40 50 70 80 90

☐ C) 90 80 70 50 40 30 10

☐ D) 70 50 40 30 20 10 0

Did you like the problem?



Submit

Ctrl + M

Next

Statement

Given an array $arr[]$ of N integers and an integer K . The task is to find the number of subarray with a maximum value equal to K .

Input Format

- The first line contains two integers N , K - the size of arr and the maximum value that should be in the subarray.
- The second line contains n integers $arr[1], arr[2], \dots, arr[n]$ — the elements of arr .

Output Format

Print the number of subarrays having K as the maximum value.

Constraints

- $1 \leq N, K \leq 10000$
- $0 \leq arr[i] \leq 10000$

Sample 1:

Input	Output
4 3 2 1 3 4	3

Explanation:

There are 3 subarrays in which the maximum element is 3, they are [2,1,3], [1,3], [3].

Sample 2:

C

```
1 #include <stdio.h>
2
3 int countSubarrays(int arr[], int n, int k) {
4     // write your code here
5     hcyh
6 }
7
8 int main() {
9     int n, k;
10
11     scanf("%d", &n);
12     scanf("%d", &k);
13
14     int arr[n];
15
16     for (int i = 0; i < n; i++) {
17         scanf("%d", &arr[i]);
18     }
19
20     int count = countSubarrays(arr, n, k);
21     printf("%d", count);
22     return 0;
23 }
24
25
26
27
```

Test against Custom Input

4 3
2 1 3 4

Ctrl + K

Run

Submit

Next

FULL HD 1080

acer



Saved



Senthamizh Selvan

57 Min : 54 Sec Prev 17/30 Next

Statement

Consider the following C function:

```
struct Node {
    int data;
    struct Node* next;
};

int detectCycle(struct Node* head) {
    struct Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return 1; // Cycle detected
    }
    return 0;
}
```

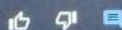
What is the time complexity of detecting the cycle in terms of the length of the straight part of the linked list (L) and the length of the cycle (C)?

Select the correct answer

☒ A) $O(L + C)$ ☐ B) $O(L * C)$ ☐ C) $O(L^2)$ ☐ D) $O(C^2)$

Did you like the problem?

2 users found this helpful



Ctrl + /

Submit

Next

FULL HD 1080

acer

Statement

You are given an integer n and m , representing the dimensions of an $n \times m$ matrix. You need to construct an $n \times m$ matrix such that the following properties are satisfied:

- Each element in the matrix is 1.
- The sum of the elements of the matrix is even.

If it is not possible then print -1.

Input Format

- The first line contains one integer n and m , the size of the matrix.

Output Format

- Print a 2d matrix with given properties or -1.

Constraints

- $1 \leq n, m \leq 100$

Sample 1:

Input	Output
2 2	1 1 1 1

Explanation:

The sum of elements of the matrix is 4 which is even.

Sample 2:

C

```
1 #include <stdio.h>
2
3 int main() {
4     // Write your code here
5     fcfh
6     return 0;
7 }
```

Test against Custom Input

2 2

Run

Submit

Next

FULL HD 1080

acer



Saved



Senthamizh Selvan

57

Min :

41

Sec

<

Prev

19 / 30

Next

>

Statement

What is the time complexity of the sliding_window function?

```
int* sliding_window(int* nums, int numsSize, int k, int* returnSize) {
    struct Deque* dq = createDeque(k);
    int* result = (int*)malloc((numsSize - k + 1) * sizeof(int));
    int resultIdx = 0;

    for (int i = 0; i < numsSize; ++i) {
        while (!isEmpty(dq) && nums[front(dq)] < nums[i]) pop_front(dq);

        push_back(dq, i);
        if (i >= k - 1) {
            result[resultIdx++] = nums[front(dq)];
            if (front(dq) == i - k + 1) pop_front(dq);
        }
    }
    *returnSize = resultIdx;
    return result;
}
```

Assume that `Deque` structure is implemented to represent deque data structure.

Select the correct answer

☒ A) $O(n)$ ☐ B) $O(nk)$ ☐ C) $O(n \log n)$ ☐ D) $O(n \log k)$

Did you like the problem?

1 user found this helpful



Ctrl + J

Submit

Next

FULL HD 1080

acer

Statement

You are given an array *arr* of *N* positive integers. In each move, you can pick two different numbers from the array where their absolute difference is at most one, and remove the smaller one. If two numbers are the same, you can remove either. Your goal is to determine if you can reduce the array to exactly one number using these moves.

Function Declaration

Function Name

canReduce – This function determines whether the array can be reduced to just one number based on the given rules.

Parameters

N : An integer representing the size of the array.
arr : An array of integers representing the given sequence.

Return Value

Returns a boolean: *true* if the array can be reduced to a single number, and *false* otherwise.

Constraints:

$1 \leq N \leq 10^4$
 $1 \leq arr[i] \leq 10^4$

The input and output formats provided below are only for testing with custom inputs. You only need to return the value. Printing is handled automatically.

Input Format

- The first line contains an integer *N* representing the size of *arr*.

C

```
1 bool canReduce(int N, int* arr) {
2     //write your code here
3     jfhtv
4 }
```

Test against Custom Input

```
4
4 1 3 2
```

Ctrl + Enter

Run

Submit

Next

FULL HD 1080

acer



Saved



Senthamizh Selvan

57 Min : 34 Sec Prev 21 / 30 Next

Statement

Consider the following code for prefix sum computation and range sum query:

```
void computePrefixSum(int arr[], int n, int prefixSum[]) {
    prefixSum[0] = arr[0];
    for (int i = 1; i < n; i++) {
        prefixSum[i] = prefixSum[i - 1] + arr[i];
    }
}

int rangeSumQuery(int prefixSum[], int l, int r) {
    if (l == 0)
        return prefixSum[r];
    else
        return prefixSum[r] - prefixSum[l - 1];
}
```

What is the time complexity of answering a single range sum query using the rangeSumQuery function?

Select the correct answer

☐ A) $O(n)$

☐ B) $O(\log n)$

☒ C) $O(1)$

☐ D) $O(n^2)$

Did you like the problem?



Submit

Next

FULL HD 1080

acer

Statement

Given an $n \times n$ matrix, print the elements of the matrix in an anticlockwise spiral order starting from the top-left element. For example, given the following matrix

```
| 1  2  3  4 |
| 5  6  7  8 |
| 9 10 11 12 |
|13 14 15 16 |
```

The output should be: [1,5,9,13,14,15,16,12,8,4,3,2,6,10,11,7]

Input Format

- The first line contains a single integer n the size of a 2d square matrix.
- The next n lines contain n space separated elements - describing the i th row of the matrix.

Output Format

Print the matrix in spiral order as a list separated by space.

Constraints

- $1 \leq n \leq 100$.
- $1 \leq \text{matrix}[i][j] \leq 10000$.

Sample 1:

Input	Output
4 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	1 5 9 13 14 15 16 12 8 4 3 2 6 10 11 7

C

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int* spiralOrderAnticlockwise(int** matrix, int n) {
5     // write your code here
6     int* result = (int*)malloc(n * sizeof(int));
7     int i = 0;
8
9     int main() {
10         int n;
11         scanf("%d", &n);
12
13         int** matrix = (int**)malloc(n * sizeof(int*));
14
15         for (int i = 0; i < n; ++i) {
16             matrix[i] = (int*)malloc(n * sizeof(int));
17             for (int j = 0; j < n; ++j) {
18                 scanf("%d", &matrix[i][j]);
19             }
20         }
21
22         int* result = spiralOrderAnticlockwise(matrix, n);
23
24         for (int i = 0; i < n * n; ++i) {
25             printf("%d ", result[i]);
26         }
27         printf("\n");
28         free(result);
29
30         return 0;
31     }
```

Test against Custom Input

```
4
1 2 3 4
5 6 7 8
9 10 11 12
```

Ctrl + Enter

Run

Submit

Next

FULL HD 1080

acer



Saved



Senthamizh Selvan

57

Min :

27

Sec



Prev 23 / 30



Next

Statement

Using the Master Theorem, what is the time complexity $T(n)$ of the complexFunction? (Consider all the imports are included)

```
int complexFunction(n){  
    if (n <= 1)  
        return 1;  
    else  
        return 4 * complexFunction(n / 3) + pow(n,2);  
}
```

Select the correct answer

☐ A) $T(n) = O(n^2)$

☐ B) $T(n) = O(n^2 \log n)$

☒ C) $T(n) = O(n^3)$

☐ D) $T(n) = O(n^3 \log n)$

Did you like the problem?



Submit

0/100

Next

FULL HD 1080

acer

Statement

You are given a list of integers. Your task is to sort the list based on the frequency of each integer in ascending order. If two integers have the same frequency, the one with the smaller value should come first.

Input Format

- The first line contains a single integer n , the number of integers in the list.
- The second line contains n space separated integers $a_1, a_2, \dots, a_n$.

Output Format

Print the sorted list based on the frequency of each integer.

Constraints

- $1 \leq n \leq 100$.
- $1 \leq a_i \leq 100$.

Sample 1:

Input	Output
4 1 2 3 2	1 3 2 2

Explanation:

The frequencies are: 1 appears once, 2 appears twice, and 3 appears once.

Sorting primarily by frequency, integers 1 and 3 (each appearing once) come before 2 (appearing twice). Since 1 and 3 have the same frequency, they are sorted by their value, resulting in 1 before 3.

C

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void frequencyBasedSort(int n, int arr[]) {
5     // Write your code here
6     nkhubn
7 }
8
9 int main() {
10     int n;
11     scanf("%d", &n);
12
13     int arr[n];
14     for (int i = 0; i < n; ++i) {
15         scanf("%d", &arr[i]);
16     }
17
18     // Function call to perform frequency-based sorting
19     frequencyBasedSort(n, arr);
20
21     return 0;
22 }
    
```

Test against Custom Input

4
1 2 3 2

Ctrl + Enter

Run

Submit

Next

FULL HD 1080

acer

Statement

Given a positive integer c , decide whether it can be represented as sum of two square numbers that is $c = a^2 + b^2$ or not, where a and b are two non-negative integers and $a \leq b$.

Input Format

First line contains positive integers c .

Output Format

Print **true** if it can be represented, else print **false**.

Constraints

- $1 \leq c \leq 10^9$

Sample 1:

Input	Output
25	true

Explanation:

25 can be represented in form of $3^2 + 4^2 = 9 + 16 = 25$.

Sample 2:

Input	Output
15	false

C

```

1 #include <stdio.h>
2 #include <stdbool.h>
3 #include <math.h>
4
5 bool checkSquareSum(long long int c) {
6     // Write your code here
7     nkbn
8 }
9
10 int main() {
11     long long int c;
12     scanf("%lld", &c);
13
14     if (checkSquareSum(c)) {
15         printf("true\n");
16     } else {
17         printf("false\n");
18     }
19
20     return 0;
21 }
    
```

Test against Custom Input

25

Ctrl + J

Run

Submit

Reset

FULL HD 1080

acer

Statement

You are given a set of integers represented by an array `arr`, which may contain duplicate elements. Your task is to write a recursive function to generate all unique subsets of this array. Each subset should be unique, even if the array contains duplicate elements. The solution should not generate any duplicate subsets.

A subset of a set is a collection of elements that are all derived from the original set. Subsets can range from the empty set (containing no elements) to the set itself (containing all elements).

Function Declaration

Function Name

`uniqueSubsets` - This function generates all unique subsets of a given integer array that may contain duplicate values.

Parameters

- `multiset` : A reference to an integer array representing the elements of the set.

Return Value

- Returns a 2D array (list of lists) containing all unique subsets of the given array.
- Each subset should appear exactly once, even if the input array contains duplicate elements.

Constraints:

- $1 \leq n \leq 10$
- $1 \leq arr[i] \leq 25$

Input Format

- The first line contains one integer `n`, the size of the array. - Next line contains `n` integers `arr[0], arr[1]...arr[n]`, representing the elements of the array.

C

```
1- void uniqueSubsets(int* multiset, int n, int*** subsets, int** subsetSizes, int* subsetCount)
2- {
3-     // Write your code here
4-     khbhf
5- }
6-
```

Test against Custom Input

```
2
1 2
```

Ctrl + Z

Run

Submit

Next

FULL HD 1080

acer

Statement

What of the given sorting snippet uses repeatedly compares and swaps adjacent elements if they are in the wrong order, causing the largest elements to "bubble up" to their correct positions? Consider all imports are included)

Select the correct answer

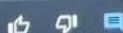
A)

```
void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int min_idx = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

B)

```
void sort(int arr[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Did you like the problem?



Submit

0/1/2

Next

FULL HD 1080

acer

Statement

You are given a list of integers *heights* representing the heights of *N* mountain peaks. For each peak, you need to find the height of the next higher peak that comes after it in the list. If there is no higher peak after a given peak, return -1 for that peak.

Input Format

- The first line contains one integer *n*, the size of the list of integers. - Next line contains *n* integers *heights*[0], *heights*[1]...*heights*[*n*], representing the peaks.

Output Format

- For each peak, print the next higher peak that comes after it in the list. If there is no higher peak for a peak than print -1.

Constraints

- $1 \leq n \leq 100000$
- $0 \leq heights[i] \leq 1000000$

Sample 1:

Input	Output
2 1 2	2 -1

Explanation:

The next greater element for 1 is 2 and for 2 there is no next greater element so for 2 it will be -1.

Sample 2:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Function to find the next higher peak for each element in heights
5 int* next_higher_peak(int* heights, int n) {
6     // Write your code here
7     jhtfvjv
8 }
9
10 int main() {
11     int n;
12     // Take the size of the heights array as input
13     scanf("%d", &n);
14
15     int* heights = (int*)malloc(n * sizeof(int));
16
17     // Take the heights array elements as input
18     for (int i = 0; i < n; ++i) {
19         scanf("%d", &heights[i]);
20     }
21
22     // Get the next higher peaks
23     int* result = next_higher_peak(heights, n);
24
25     // Print the result
26     for (int i = 0; i < n; ++i) {
27         printf("%d ", result[i]);
28     }
29
30     // Free allocated memory for heights and result arrays
31     free(heights);
32     free(result);
33 }
```

Test against Custom Input

2
1 2

Run Submit Next

FULL HD 1080p

acer

Statement

What will be the value of the result, if construct_algo function is called with a value 5? (Consider all imports are included)

```
int* construct_algo(int num) {  
    int* result = (int*)malloc(num * sizeof(int));  
    if (result == NULL) {  
        exit(EXIT_FAILURE);  
    }  
  
    for (int i = 0; i < num; ++i) {  
        if (i == 0 || i == 1) result[i] = i;  
        else result[i] = result[i - 1] + result[i - 2];  
    }  
  
    return result;  
}
```

Select the correct answer

- ☒ A) {0, 1, 1, 2, 3}
- ☐ B) {0, 1, 1, 2, 4}
- ☐ C) {0, 1, 2, 3, 5}
- ☐ D) {0, 1, 3, 6, 10}

Did you like the problem?
1 user found this helpful



Submit

Next

FULL HD 1080

acer



Saved



Senthamizh Selvan

57

Min :

06

Sec



Prev

30 / 30

View Summary



Statement

Given the following square 2D matrix $mat[3][3] = \{ \{5, 1, 9\}, \{2, 4, 8\}, \{7, 6, 3\} \}$:

```
void fun1(int mat[N][N]) {  
    for (int x = 0; x < N / 2; x++) {  
        for (int y = x; y < N - x - 1; y++) {  
            int temp = mat[x][y];  
            mat[x][y] = mat[y][N - 1 - x];  
            mat[y][N - 1 - x] = mat[N - 1 - x][N - 1 - y];  
            mat[N - 1 - x][N - 1 - y] = mat[N - 1 - y][x];  
            mat[N - 1 - y][x] = temp;  
        }  
    }  
}
```

What will be the updated matrix after calling `fun1(mat)`? (Consider all imports are included)

Select the correct answer

☐ A) 3 8 9
6 4 1
7 2 5

☐ B) 9 8 3
1 4 6
5 2 7

☐ C) 7 6 3
2 4 8
5 1 9

☒ D) 7 2 5
6 4 1
3 8 9

Did you like the problem?



Submit

View Summary

FULL
HD 1080

acer

Data structures and Algorithms in C test

59 Min : 38 Sec

30 / 30

 Total attempted

30

☒ Total Attempted

00

☐ Unattempted

[Back to Assessment](#)

[Submit and Finish](#)

Full HD 1080p

acer

Data structures and Algorithms in C test - Report

Next attempt again after 29 Days

Your Total Score

1400 / 3000 points

14 Correct Answers 16 Wrong Answers 00 Unattempted

[Share test score](#)
[View Solutions](#)

46%

SKILL SCORE

Topic wise summary

Topic	Correct	Total	Remarks	Resources
Time complexity	1	2	Weak	Learn →
Two pointers	2	5	Weak	
Queues	3	3	Good	Learn →
Arrays	1	3	Weak	Learn →
Prefix Sum	1	3	Weak	Learn →

Arrays	1	3	• Weak	Learn →
Prefix Sum	1	3	• Weak	Learn →
Stacks	1	2	• Weak	Learn →
Recursion	2	4	• Weak	Learn →
Matrix	1	4	• Weak	Learn →
LinkedLists	1	1	• Good	Learn →
Sorting	1	3	• Weak	Learn →

Problem Type

Type	Correct	Total
MCQs	12	15
Coding Problems	2	15

Did you like the assessment ?  

